MODULE 21 · DOMAIN 4 - ASSEMBLING AND DEPLOYING APPLICATIONS

AI Search: the moving parts

Endpoints, indexes, embeddings, sync and queries: what each piece does, and how to create and query an index.

SECTION 4 of 6 EXAM WEIGHT 22% MODULE 21 of 40

admin@sthithapragna.com

AI Search is a library. The endpoint is the building, the index is the catalogue, the Delta table is the shelves, and sync is the librarian keeping the catalogue current.

Four things you create or choose

PART	WHAT IT IS
Endpoint	Compute that hosts indexes and serves queries. Standard or storage-optimised
Source Delta table	The chunk table from module 10: primary key, text, metadata
Index	Vectors plus synced columns, governed in Unity Catalog
Embedding model	A serving endpoint that turns text into vectors, or your own vectors

ON THE EXAM

One endpoint can host many indexes. The index name is a three-level Unity Catalog name.

Delta Sync index, or Direct Vector Access index?

DELTA SYNC

Kept in step with a table

- Reads a source Delta table
- Updates as the table changes
- Managed or self-managed embeddings
- The usual choice for rag

DIRECT VECTOR ACCESS

You write it yourself

- No source table
- Upsert and delete through the SDK or REST
- You supply every vector
- Cannot be created in the UI

THE TIEBREAKER

If the content is in, or can go into, a Delta table, use Delta Sync. Direct Vector Access is for vectors your own system writes directly.

Managed embeddings, or self-managed?

MANAGED

Databricks embeds for you

- embedding_source_column: the text
- embedding_model_endpoint_name
- Query with query_text
- Same model used for queries automatically

SELF-MANAGED

You embed, you store

- embedding_vector_column: your vectors
- embedding_dimension: their size
- Query with query_vector
- You must embed queries with the same model

THE TIEBREAKER

query_text works when Databricks manages embeddings. With your own vectors, embed the query yourself and pass query_vector.

Triggered sync, or continuous?

TRIGGERED

Sync when you say

- Start with index.sync() or the API
- Cheaper: no always-on pipeline
- Right for scheduled updates
- The only mode on storage-optimised

CONTINUOUS

Always in step

- Seconds of latency
- A streaming pipeline runs all the time
- Higher cost
- Standard endpoints only

THE TIEBREAKER

If the source changes on a schedule, trigger a sync after each load. Pay for continuous only when minutes of staleness matter.

create_delta_sync_index: key parameters

PARAMETER	PURPOSE
`endpoint_name, index_name`	Where it lives; three-level name
`source_table_name, primary_key`	The Delta table and its key
`pipeline_type`	TRIGGERED or CONTINUOUS
`embedding_source_column`	Text to embed, plus model endpoint
`embedding_vector_column`	Your vectors, plus dimension
`columns_to_sync`	Metadata for filters and citations

ON THE EXAM

Managed: source column plus model endpoint. Self-managed: vector column plus dimension.

From a chunk table to a queryable index

01
Endpoint
Create it, standard or storage-optimised.

02
Table
Primary key, and change data feed on.

03
Index
create_delta_sync_index.

04
Sync
Wait until ready, or trigger a sync.

05
Query
similarity_search from code or SQL.

ON THE EXAM

A standard endpoint needs change data feed on the source table. Without it, the index cannot follow changes.

similarity_search: the arguments

ARGUMENT	PURPOSE
query_text	Plain text, for managed embeddings or hybrid and full-text search
query_vector	A vector you computed, for self-managed embeddings
columns	Which columns to return with each result
num_results	How many results to return
filters	Restrict results by metadata
query_type	ANN by default, or HYBRID, or FULL_TEXT

ON THE EXAM

Hybrid adds keyword matching to vectors, and returns at most 200 results. Full-text is keyword only, and in Beta.

Filters differ by endpoint type

STANDARD ENDPOINT

A Python dictionary

- Key holds column and operator
- {"price <": 200}
- Several keys combine with AND
- Array filters supported

STORAGE-OPTIMISED ENDPOINT

A SQL-like string

- Like a WHERE clause
- "price < 200 AND region = 'EU'"
- No array filtering
- Filters after over-fetching

THE TIEBREAKER

A dictionary filter on a storage-optimised endpoint fails. On storage-optimised, a strict filter can even return nothing.

Other facts the exam can ask about

- Search uses **HNSW**, an approximate nearest neighbour algorithm, with **L2 distance**.
- Normalise vectors if you want cosine similarity. GTE's are not normalised.
- A SQL function, **vector_search()**, queries an index from SQL.
- Use **service principals**, not personal access tokens, in production.
- Old names survive in code: REST paths say vector-search, and LangChain says DatabricksVectorSearch.

> ON THE EXAM
>
> Product renames rarely reach every API path at once. Expect both names in real code and on the exam.

Harbourline's policy index

01
Endpoint
Standard: 2 million chunks, low latency.

02
Table
chunk_id key, change data feed on.

03
Index
Delta Sync, managed GTE embeddings.

04
Sync
Triggered after the nightly policy load.

05
Query
Hybrid, dictionary filter on product and year.

ON THE EXAM

Policies change nightly, so a triggered sync after each load costs far less than continuous, for the same freshness.

> A team's source table is refreshed once a night by a batch job. Its Delta Sync index on a standard endpoint uses CONTINUOUS sync.
>
> Finance asks why AI Search costs are so high.

What should the engineer change?

A Keep continuous sync, but reduce the number of synced columns

B Switch to triggered sync, and sync after each nightly load

C Switch to a Direct Vector Access index written by hand

D Move to a storage-optimised endpoint with continuous sync

◆ PAUSE AND DECIDE

What should the engineer change?

A Fewer columns, still continuous

The always-on pipeline is the cost, not the columns.

B Triggered, after each load

Same freshness for nightly data, no always-on pipeline.

C Direct Vector Access

Replaces automatic sync with hand-written code.

D Storage-optimised, continuous

Not possible: storage-optimised is triggered only.

THE SIGNAL

Refreshed once a night, but synced continuously. Paying for seconds of freshness that the data never needs.

> An index was created with embedding_vector_column and embedding_dimension, because the team computes its own vectors.
>
> When the engineer calls similarity_search with query_text, the query fails.

What should the engineer do?

A Recreate the index on a storage-optimised endpoint instead

B Add the embedding model's name to the columns argument

C Set query_type to HYBRID so that plain text is accepted

D Embed the query with the same model, then pass query_vector

◆ PAUSE AND DECIDE

What should the engineer do?

A Storage-optimised endpoint

Endpoint type does not change who owns the embeddings.

B Model name in columns

columns chooses returned fields. It configures nothing.

C HYBRID query type

The vector half still needs an embedding Databricks cannot make.

D Embed, then query_vector

Self-managed means you embed queries too, with the same model.

THE SIGNAL

embedding_vector_column means self-managed. Databricks has no model to embed your text, so you pass the vector.

> A fraud system produces a vector for each transaction in its own streaming service. The vectors never land in a Delta table.
>
> The system must insert and delete vectors in the index itself, as transactions arrive and expire.

Which kind of index fits?

A. A Direct Vector Access index, written through the SDK

B. A Delta Sync index with continuous sync from a table

C. A Delta Sync index using managed embeddings on text

D. A dedicated full-text index on a storage-optimised endpoint

◆ PAUSE AND DECIDE

Which kind of index fits?

A Direct Vector Access

No source table; the system upserts and deletes its own vectors.

B Delta Sync, continuous

Needs a source table, which does not exist here.

C Delta Sync, managed embeddings

Needs a table of text. The system already has vectors.

D Full-text index

Keyword only. These are vectors, not text.

THE SIGNAL

Never land in a Delta table, and the system writes vectors itself. That is exactly what Direct Vector Access is for.

> An index on a storage-optimised endpoint must return only EU documents. The engineer passes filters={"region": "EU"} to similarity_search.
>
> The query fails with a filter error.

What should the engineer change?

A Pass the filter as a list of column names instead

B Put the word EU into the query_text instead of a filter

C Use a SQL-like filter string such as region = 'EU'

D Switch to the HYBRID query type so the filter works

◆ PAUSE AND DECIDE

What should the engineer change?

A List of column names

Not a filter format on either endpoint type.

B EU in query_text

Nudges similarity. Does not guarantee EU-only results.

C SQL-like filter string

Storage-optimised endpoints take a WHERE-style string.

D HYBRID query type

Changes how matching works, not the filter syntax.

THE SIGNAL

A dictionary filter on a storage-optimised endpoint. Dictionaries are for standard endpoints; storage-optimised wants a SQL-like string.

What to carry forward

- Parts: **endpoint, source table, index, embedding model**.
- **Delta Sync** follows a table. **Direct Vector Access** is written by your code.
- Managed: query_text. Self-managed: **embed yourself, pass query_vector**.
- Triggered for scheduled data. Continuous costs more. Storage-optimised: triggered only.
- Filters: **dictionary on standard**, **SQL-like string on storage-optimised**.

> ON THE EXAM
>
> Next: module 22 sizes AI Search for a workload: standard or storage-optimised, and when to switch things off.